



Guía Completa: Despliegue VPS Contabo con Multi-Apps

De Docker a Producción: Django Ecodisseny + Oscar Shop + VSCode Remote

Índice Completo



FASE 1: PREPARACIÓN

- [1.1 Preparación Pre-Migración](#)
- [1.2 Backup y Configuración](#)



FASE 2: VPS CONTABO

- [2.1 Contratación y Especificaciones](#)
- [2.2 Configuración Inicial del Sistema](#)
- [2.3 Instalación de Dependencias](#)



FASE 3: MIGRACIÓN DE DATOS

- [3.1 Clonación del Repositorio](#)
- [3.2 Configuración con Scripts Automatizados](#)
- [3.3 Transferencia de Datos](#)



FASE 4: CONFIGURACIÓN DE PRODUCCIÓN

- [4.1 Django Settings de Producción](#)
- [4.2 Nginx y Proxy Reverso](#)
- [4.3 Gunicorn como Servicio](#)



FASE 5: DNS Y SSL

- [5.1 Configuración DNS Cloudflare](#)
- [5.2 Certificados SSL Let's Encrypt](#)
- [5.3 Optimizaciones Cloudflare](#)



FASE 6: DESARROLLO REMOTO

- [6.1 VSCode Remote SSH](#)
- [6.2 Configuración de Desarrollo](#)
- [6.3 Debugging y Testing](#)



FASE 7: MULTI-APLICACIONES

- [7.1 Arquitectura Multi-App](#)
- [7.2 Configuración Django Oscar](#)
- [7.3 Gestión de Puertos](#)

 FASE 8: MONITOREO Y MANTENIMIENTO

- [8.1 Scripts de Monitoreo](#)
- [8.2 Backups Automatizados](#)
- [8.3 Scripts de Actualización](#)

 FASE 9: TROUBLESHOOTING

- [9.1 Problemas Comunes](#)
- [9.2 Debugging Avanzado](#)
- [9.3 Checklist de Verificación](#)

 FASE 1: PREPARACIÓN

1.1 Preparación Pre-Migración

Checklist de Verificación

```
# En tu entorno Docker actual:
✓ Verificar que la aplicación funciona correctamente
✓ Backup completo de la base de datos
✓ Backup de archivos media
✓ Documentar configuraciones específicas
✓ Preparar variables de entorno de producción
✓ Validar que todos los tests pasan
```

1.2 Backup y Configuración

Generar Backup Completo

```
# Asegúrate de que Docker está funcionando
docker-compose up -d

# Backup de base de datos
docker-compose exec db pg_dump -U postgres ecodisseny >
backup_desarrollo_$(date +%Y%m%d).sql

# Backup de archivos media
tar -czf media_backup_$(date +%Y%m%d).tar.gz media/

# Listar apps y dependencias
docker-compose exec web pip freeze > requirements_frozen.txt
```

Preparar Variables de Entorno

```
# Crear archivo .env de producción
cat > .env.production << EOF
DEBUG=False
SECRET_KEY=GENERAR_UNA_CLAVE_SUPER_SECRETA_AQUI_64_CARACTERES_MINIMO
DATABASE_URL=postgresql://deploy:PASSWORD_SEGURO@localhost:5432/ecodisseny_
prod
ALLOWED_HOSTS=app.arasmu.net,tu-ip-vps
SECURE_SSL_REDIRECT=True
SECURE_PROXY_SSL_HEADER=HTTP_X_FORWARDED_PROTO,https
EOF
```

FASE 2: VPS CONTABO

2.1 Contratación y Especificaciones

VPS Contabo Cloud VPS 10

```
📄 Especificaciones:
├─ CPU: 3 vCPU Cores
├─ RAM: 8 GB
├─ Storage: 75 GB NVMe
├─ Región: European Union
├─ Precio: €3.60/mes (€43.20/año)
└─ Ancho de banda: 32TB

🎯 Capacidad vs Uso Proyectado:
├─ Django Ecodisseny: ~200MB RAM
├─ Django Oscar Shop: ~200MB RAM
├─ PostgreSQL: ~300MB RAM
├─ Nginx + Sistema: ~500MB RAM
└─ Total estimado: ~1.2GB de 8GB (sobra mucho!)
```

2.2 Configuración Inicial del Sistema (Simplificada para Docker)

Primer Acceso y Setup Básico

```
# Conectar por primera vez (Contabo te enviará credenciales)
ssh root@TU_IP_VPS

# Actualizar sistema
apt update && apt upgrade -y

# Instalar dependencias mínimas para Docker
apt install -y docker.io docker-compose git nginx certbot python3-certbot-
nginx htop curl wget
```

```
# Opcional: Cambiar puerto SSH por seguridad
nano /etc/ssh/sshd_config
# Cambiar: Port 22 → Port 2222
systemctl restart sshd

# Configurar firewall
ufw allow 2222    # SSH (si cambiaste puerto)
ufw allow 22     # SSH (si mantienes puerto por defecto)
ufw allow 80     # HTTP
ufw allow 443    # HTTPS
ufw --force enable

# Verificar que Docker funciona
docker --version
docker-compose --version
```

⚠ Importante - Docker vs Manual:

Con **Docker** ya NO necesitas:

- × Python3, pip, venv (Docker se encarga)
- × PostgreSQL nativo (usamos container)
- × Usuario deploy (puedes usar root para Docker)
- × Configuración manual de Python

Solo necesitas: Docker + Nginx nativo + Certbot



FASE 3: DESPLIEGUE CON DOCKER

3.1 Clonar y Configurar Proyecto

```
# Clonar repositorio
git clone https://github.com/Mulastone/ecodisseny_dj_pg.git
cd ecodisseny_dj_pg

# El proyecto ya incluye:
# ✔ docker-compose.yml optimizado para VPS
# ✔ nginx/vps-app.arasmu.net.conf (configuración Nginx)
# ✔ deploy-vps.sh (script automatizado)
```

3.2 Despliegue Automatizado

```
# Hacer ejecutable el script
chmod +x deploy-vps.sh

# Ejecutar despliegue completo
./deploy-vps.sh
```

El script **deploy-vps.sh** hace automáticamente:

1.  Actualiza código desde Git
2.  Para contenedores existentes
3.  Construye nueva imagen Docker
4.  Inicia servicios (PostgreSQL + Django)
5.  Configura Nginx automáticamente
6.  Verifica que todo funciona

3.3 Configuración Manual de Nginx (si es necesario)

```
# Si el script no configuró Nginx automáticamente:
sudo cp nginx/vps-app.arasmu.net.conf /etc/nginx/sites-
available/app.arasmu.net
sudo ln -sf /etc/nginx/sites-available/app.arasmu.net /etc/nginx/sites-
enabled/
sudo rm -f /etc/nginx/sites-enabled/default

# Verificar configuración
sudo nginx -t
sudo systemctl reload nginx
```

🌐 FASE 4: CONFIGURACIÓN DNS Y SSL

2.3 Instalación de Dependencias

```
# Instalar dependencias del sistema
sudo apt install -y \
    python3 \
    python3-pip \
    python3-venv \
    python3-dev \
    build-essential \
    libpq-dev \
    postgresql \
    postgresql-contrib \
    nginx \
    certbot \
    python3-certbot-nginx \
    htop \
    curl \
    wget \
    unzip

# Configurar PostgreSQL
sudo -u postgres createuser --interactive deploy
sudo -u postgres createdb -O deploy ecodisseny_prod
```

```
sudo -u postgres psql -c "ALTER USER deploy PASSWORD
'tu_password_postgresql_seguro';"

# Crear estructura de directorios
mkdir -p /home/deploy/{ecodisseny,oscar_shop,scripts,backups,logs}
mkdir -p /var/log/django
sudo chown deploy:deploy /var/log/django
```

FASE 3: MIGRACIÓN DE DATOS

3.1 Clonación del Repositorio

```
cd /home/deploy/ecodisseny

# Clonar desde GitHub
git clone https://github.com/Mulastone/ecodisseny_dj_pg.git .

# Crear entorno virtual
python3 -m venv venv
source venv/bin/activate

# Instalar dependencias
pip install -r requirements.txt
pip install gunicorn psycopg2-binary whitenoise
```

3.2 Configuración con Scripts Automatizados

✂ Uso de Scripts de Configuración del Proyecto

```
cd /home/deploy/ecodisseny
source venv/bin/activate

# Configurar variables de entorno
cat > .env << 'EOF'
DEBUG=False
SECRET_KEY=tu_secret_key_super_segura_de_64_caracteres_minimo_aqui
DATABASE_URL=postgresql://deploy:tu_password_postgresql_seguro@localhost:54
32/ecodisseny_prod
ALLOWED_HOSTS=app.arasmu.net,tu-ip-vps
SECURE_SSL_REDIRECT=True
SECURE_PROXY_SSL_HEADER=HTTP_X_FORWARDED_PROTO,https
EOF

# Opción 1: Configuración completa automática (RECOMENDADO)
chmod +x setup_complete.sh
./setup_complete.sh

# Esto ejecuta automáticamente:
```

```
# ✔ Reinicio de migraciones
# ✔ Configuración de base de datos
# ✔ Carga de fixtures maestros
# ✔ Creación de usuarios y perfiles

# Usuarios creados automáticamente:
echo "👑 ADMIN - mulastone: ecodisseny2024"
echo "👑 ADMIN - gonzalo: ecodisseny2024"
echo "👤 USER - sarah: ecodisseny2024"
echo "👤 USER - pilar: ecodisseny2024"
echo "👤 USER - santiago: ecodisseny2024"
echo "👤 USER - roger: ecodisseny2024"

# Cargar documentación y configurar cache
python manage.py cargar_documentacion
python manage.py createcachetable
python manage.py collectstatic --noinput
```

3.3 Transferencia de Datos

```
# En tu máquina local (transferir backups):
scp backup_desarrollo_*.sql deploy@tu-ip-vps:/home/deploy/backups/
scp media_backup_*.tar.gz deploy@tu-ip-vps:/home/deploy/backups/

# En el VPS (restaurar si necesario):
# psql ecodisseny_prod < /home/deploy/backups/backup_desarrollo_*.sql
# tar -xzf /home/deploy/backups/media_backup_*.tar.gz
```



FASE 4: CONFIGURACIÓN DE PRODUCCIÓN

4.1 Django Settings de Producción

```
# Crear ecodisseny/settings_production.py
import os
from .settings import *

# Configuración de producción
DEBUG = False
SECRET_KEY = os.getenv('SECRET_KEY')
ALLOWED_HOSTS = os.getenv('ALLOWED_HOSTS', '').split(',')

# Base de datos
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'ecodisseny_prod',
        'USER': 'deploy',
        'PASSWORD': os.getenv('DB_PASSWORD'),
```

```
        'HOST': 'localhost',
        'PORT': '5432',
        'OPTIONS': {
            'MAX_CONNS': 20,
        }
    }
}

# Cache en base de datos (sin Redis para simplicidad)
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.db.DatabaseCache',
        'LOCATION': 'django_cache_table',
    }
}

# Archivos estáticos
STATIC_ROOT = '/home/deploy/ecodisseny/static/'
MEDIA_ROOT = '/home/deploy/ecodisseny/media/'

# Configuración de seguridad
SECURE_SSL_REDIRECT = True
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = 'DENY'

# Logging optimizado
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '{levelname} {asctime} {module} {process:d}
{thread:d} {message}',
            'style': '{',
        },
    },
    'handlers': {
        'file': {
            'level': 'INFO',
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': '/var/log/django/ecodisseny.log',
            'maxBytes': 10*1024*1024,
            'backupCount': 5,
            'formatter': 'verbose',
        },
    },
    'loggers': {
        'django': {
            'handlers': ['file'],
            'level': 'INFO',
            'propagate': True,
        },
    },
}
```



```
    },  
  }  
}
```

4.2 Nginx y Proxy Reverso

```
# Configuración Nginx para Ecodisseny  
sudo tee /etc/nginx/sites-available/app.arasmu.net << 'EOF'  
server {  
    listen 80;  
    server_name app.arasmu.net;  
    return 301 https://$server_name$request_uri;  
}  
  
server {  
    listen 443 ssl http2;  
    server_name app.arasmu.net;  
  
    # SSL será configurado por Certbot  
  
    # Security headers  
    add_header X-Frame-Options DENY always;  
    add_header X-Content-Type-Options nosniff always;  
    add_header X-XSS-Protection "1; mode=block" always;  
    add_header Strict-Transport-Security "max-age=31536000;  
includeSubDomains" always;  
  
    # Archivos estáticos  
    location /static/ {  
        alias /home/deploy/ecodisseny/static/;  
        expires 1y;  
        add_header Cache-Control "public, immutable";  
    }  
  
    location /media/ {  
        alias /home/deploy/ecodisseny/media/;  
        expires 30d;  
    }  
  
    # Django app  
    location / {  
        proxy_pass http://127.0.0.1:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
        proxy_redirect off;  
    }  
  
    # Logs  
    access_log /var/log/nginx/app.arasmu.net.access.log;  
    error_log /var/log/nginx/app.arasmu.net.error.log;
```

```

        client_max_body_size 50M;
    }
EOF

# Habilitar sitio
sudo ln -s /etc/nginx/sites-available/app.arasmu.net /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx

```

4.3 Gunicorn como Servicio

```

# Configuración Gunicorn
mkdir -p /home/deploy/ecodisseny/config
cat > /home/deploy/ecodisseny/config/gunicorn.conf.py << 'EOF'
bind = "127.0.0.1:8000"
workers = 2 # Optimizado para 8GB RAM y pocos usuarios
worker_class = "sync"
timeout = 300
max_requests = 1000
max_requests_jitter = 100
preload_app = True
accesslog = "/var/log/django/gunicorn_access.log"
errorlog = "/var/log/django/gunicorn_error.log"
loglevel = "info"
proc_name = "gunicorn_ecodisseny"
user = "deploy"
group = "deploy"
EOF

# Servicio Systemd
sudo tee /etc/systemd/system/ecodisseny.service << 'EOF'
[Unit]
Description=Ecodisseny Django App
Requires=postgresql.service
After=network.target postgresql.service

[Service]
Type=exec
User=deploy
Group=deploy
WorkingDirectory=/home/deploy/ecodisseny
Environment="PATH=/home/deploy/ecodisseny/venv/bin"
Environment="DJANGO_SETTINGS_MODULE=ecodisseny.settings_production"
EnvironmentFile=/home/deploy/ecodisseny/.env
ExecStart=/home/deploy/ecodisseny/venv/bin/gunicorn -c
/home/deploy/ecodisseny/config/gunicorn.conf.py ecodisseny.wsgi:application
ExecReload=/bin/kill -s HUP $MAINPID
Restart=always
RestartSec=10

```

```
[Install]
WantedBy=multi-user.target
EOF

# Activar servicio
sudo systemctl daemon-reload
sudo systemctl enable ecodisseny
sudo systemctl start ecodisseny
sudo systemctl status ecodisseny
```

4.1 Configuración DNS Cloudflare

Registros DNS a Añadir

```
# En Cloudflare Dashboard > DNS > Records:

❶ Aplicación Django:
  Tipo: A
  Nombre: app
  Contenido: 161.97.147.142 (tu IP VPS)
  Proxy status: DNS only (🟡 gris) - IMPORTANTE

❷ Futuro Oscar Shop:
  Tipo: A
  Nombre: tienda
  Contenido: 161.97.147.142 (tu IP VPS)
  Proxy status: DNS only (🟡 gris) - IMPORTANTE
```

Verificar Propagación DNS

```
# Verificar que DNS apunta correctamente
dig app.arasmu.net
# Debe devolver: 161.97.147.142

# Verificar que la aplicación responde
curl -I http://161.97.147.142:8000
# Debe devolver: HTTP 200 o 302
```

4.2 Certificados SSL Let's Encrypt

```
# Generar certificados SSL automáticamente
sudo certbot --nginx -d app.arasmu.net

# Si tienes múltiples dominios:
```

```
sudo certbot --nginx -d app.arasmu.net -d tienda.arasmu.net

# Verificar renovación automática
sudo systemctl enable certbot.timer
sudo systemctl start certbot.timer
sudo certbot renew --dry-run
```

Certbot modificará automáticamente:

- ✓ Configuración SSL en Nginx
- ✓ Redirección HTTP → HTTPS
- ✓ Headers de seguridad
- ✓ Renovación automática

4.3 Verificación Final

```
# Verificar que HTTPS funciona
curl -I https://app.arasmu.net
# Debe devolver: HTTP 200

# Verificar redirección HTTP → HTTPS
curl -I http://app.arasmu.net
# Debe devolver: HTTP 301 → HTTPS

# Verificar certificado SSL
openssl s_client -connect app.arasmu.net:443 -servername app.arasmu.net
```

FASE 5: DESARROLLO REMOTO (OPCIONAL)

5.1 VSCode Remote SSH

Configurar SSH Local

```
# En tu máquina local, editar ~/.ssh/config:
Host ecodisseny-vps
  HostName 161.97.147.142
  User root # 0 deploy si prefieres
  Port 2222 # Si cambiaste el puerto SSH
  IdentityFile ~/.ssh/id_rsa
  ServerAliveInterval 60
  ServerAliveCountMax 3
```

Conectar VSCode

```
# 1. Instalar extensión "Remote - SSH" en VSCode
# 2. F1 > "Remote-SSH: Connect to Host"
# 3. Seleccionar "ecodisseny-vps"
# 4. ¡Listo! VSCode conectado al VPS
```

5.2 Comandos Útiles para Desarrollo

```
# Ver logs en tiempo real
docker-compose logs -f web
docker-compose logs -f db

# Acceder al container Django
docker-compose exec web bash

# Ejecutar comandos Django
docker-compose exec web python manage.py shell
docker-compose exec web python manage.py migrate
docker-compose exec web python manage.py cargar_documentacion --update

# Reiniciar solo Django (mantener DB)
docker-compose restart web

# Rebuilding tras cambios de código
./deploy-vps.sh
```

III FASE 6: MONITOREO Y MANTENIMIENTO

Nombre: admin Contenido: IP_DE_TU_VPS_CONTABO Proxy status: DNS only (● gris) - IMPORTANTE

```
#### Verificar Propagación

```bash
Verificar desde terminal local:
nslookup app.arasmu.net
dig app.arasmu.net
Debe devolver la IP de tu VPS
```

## 5.2 Certificados SSL Let's Encrypt

```
Una vez propagado el DNS:
sudo certbot --nginx -d app.arasmu.net -d tienda.arasmu.net

Configurar renovación automática
sudo systemctl enable certbot.timer
```

```
sudo systemctl start certbot.timer
sudo certbot renew --dry-run
```

## 5.3 Optimizaciones Cloudflare

```
Una vez SSL funcionando, opcionalmente cambiar a "Proxied":
- Cambiar "DNS only" a "Proxied" en Cloudflare
- SSL/TLS mode: "Full (strict)"
- Auto Minify: CSS, JS, HTML
- Brotli: ON
```

---

## FASE 6: DESARROLLO REMOTO

### 6.1 VSCode Remote SSH

#### Configurar SSH Local

```
En tu máquina local, editar ~/.ssh/config:
Host ecodisseny-vps
 HostName tu-ip-vps-contabo
 User deploy
 Port 22
 IdentityFile ~/.ssh/id_rsa
 ServerAliveInterval 60
 ServerAliveCountMax 3
```

#### Conectar VSCode

```
1. Instalar extensión "Remote - SSH" en VSCode
2. F1 > "Remote-SSH: Connect to Host"
3. Seleccionar "ecodisseny-vps"
4. ¡Listo! VSCode conectado al VPS
```

### 6.2 Configuración de Desarrollo

#### Python Interpreter y Extensiones

```
// En VSCode conectado al VPS:
{
 "python.defaultInterpreterPath":
 "/home/deploy/ecodisseny/venv/bin/python",
 "python.terminal.activateEnvironment": true,
```

```
"python.linting.enabled": true,
"python.formatting.provider": "black"
}
```

## Alias Útiles

```
Añadir a ~/.bashrc del VPS:
echo 'source /home/deploy/ecodisseny/venv/bin/activate' >> ~/.bashrc
echo 'alias cdeco="cd /home/deploy/ecodisseny"' >> ~/.bashrc
echo 'alias runserver="python manage.py runserver 0.0.0.0:8001"' >>
~/.bashrc
echo 'alias logs="sudo tail -f /var/log/django/ecodisseny.log"' >>
~/.bashrc
source ~/.bashrc
```

## 6.1 Scripts de Monitoreo para Docker

```
Script de monitoreo general
cat > /home/deploy/ecodisseny/scripts/monitor.sh << 'EOF'
#!/bin/bash
echo "=== 🖨️ ESTADO DEL SERVIDOR $(date) ==="
echo "📊 CPU y Memoria:"
echo "CPU: $(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)%
usado"
echo "RAM: $(free -h | awk 'NR==2{printf "%.1f/%.1fGB (%.1f%%)\n",
$3/1024/1024,$2/1024/1024,$3*100/$2}')"
echo "📁 Espacio: $(df -h / | tail -1 | awk '{print $3 "/" $2 " (" $5
")"}')'"

echo "🐳 Contenedores Docker:"
docker-compose ps

echo "🔧 Servicios del sistema:"
for service in nginx docker; do
 if systemctl is-active $service >/dev/null; then
 echo "✅ $service: ACTIVO"
 else
 echo "❌ $service: INACTIVO"
 fi
done

echo "🌐 Web: $(curl -s -o /dev/null -w "%{http_code}"
https://app.arasmu.net 2>/dev/null || echo 'Sin SSL')"
echo "🌐 Local: $(curl -s -o /dev/null -w "%{http_code}"
http://localhost:8000 2>/dev/null || echo 'Error')"
EOF

chmod +x /home/deploy/ecodisseny/scripts/monitor.sh
```

## 6.2 Backups Automatizados para Docker

```
Script de backup completo con Docker
cat > /home/deploy/ecodisseny/scripts/backup_docker.sh << 'EOF'
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/home/deploy/ecodisseny/backups"
PROJECT_DIR="/home/deploy/ecodisseny"

Crear directorio si no existe
mkdir -p $BACKUP_DIR

echo "📁 Iniciando backup Docker ($DATE)..."

1. Backup de volúmenes Docker
echo "📦 Backup de volúmenes Docker..."
docker run --rm -v ecodisseny_dj_pg_postgres_data:/data -v
$BACKUP_DIR:/backup alpine tar -czf /backup/postgres_data_$DATE.tar.gz -C
/data .
docker run --rm -v ecodisseny_dj_pg_static_files:/data -v
$BACKUP_DIR:/backup alpine tar -czf /backup/static_files_$DATE.tar.gz -C
/data .
docker run --rm -v ecodisseny_dj_pg_media_files:/data -v
$BACKUP_DIR:/backup alpine tar -czf /backup/media_files_$DATE.tar.gz -C
/data .

2. Backup de base de datos desde container
echo "🗄️ Backup de base de datos..."
docker-compose exec -T db pg_dump -U ecodisseny ecodisseny_db | gzip >
$BACKUP_DIR/database_$DATE.sql.gz

3. Backup del código y configuración
echo "📁 Backup del código..."
tar --exclude='__pycache__' --exclude='.git' --exclude='venv*' -czf
$BACKUP_DIR/codigo_$DATE.tar.gz -C /home/deploy ecodisseny

4. Limpiar backups antiguos (7 días)
find $BACKUP_DIR -name "*_*.tar.gz" -mtime +7 -delete
find $BACKUP_DIR -name "*_*.sql.gz" -mtime +7 -delete

echo "✅ Backup completado: $DATE"
ls -lh $BACKUP_DIR/*$DATE*
EOF

chmod +x /home/deploy/ecodisseny/scripts/backup_docker.sh

Cron para backup diario a las 2 AM
(crontab -l 2>/dev/null; echo "0 2 * * *
/home/deploy/ecodisseny/scripts/backup_docker.sh") | crontab -
```



## 6.3 Scripts de Actualización

```
Script de actualización para Docker
cat > /home/deploy/ecodisseny/scripts/actualizar_docker.sh << 'EOF'
#!/bin/bash
cd /home/deploy/ecodisseny

echo "🔄 Iniciando actualización Docker..."

Backup de seguridad
./scripts/backup_docker.sh

Actualizar código
echo "👉 Actualizando código..."
git pull origin docker

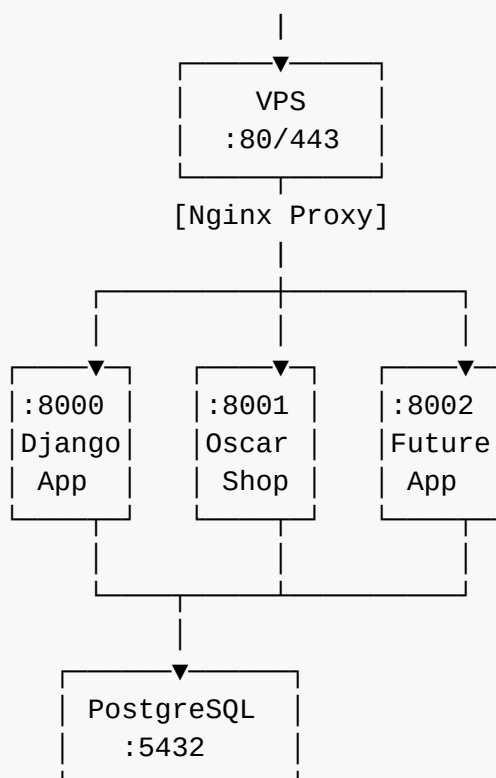
Rebuilding y reinicio
echo "🐳 Rebuilding containers..."
./deploy-vps.sh

echo "✅ Actualización completada"
EOF

chmod +x /home/deploy/ecodisseny/scripts/actualizar_docker.sh
```



## FASE 7: MULTI-APLICACIONES (PREPARACIÓN FUTURA)



### ### 7.2 Configuración Django Oscar

```
```bash
# Crear estructura para Oscar Shop
mkdir -p /home/deploy/oscar_shop
cd /home/deploy/oscar_shop

# Crear proyecto Oscar (futuro)
python3 -m venv venv
source venv/bin/activate
pip install django-oscar

# Configurar Nginx para Oscar (puerto 8001)
sudo tee /etc/nginx/sites-available/tienda.arasmu.net << 'EOF'
server {
    listen 80;
    server_name tienda.arasmu.net;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name tienda.arasmu.net;

    # SSL certificates
    ssl_certificate /etc/letsencrypt/live/tienda.arasmu.net/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/tienda.arasmu.net/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
```

7.3 Gestión de Puertos

Mapa de Puertos:

Público:

- |— Puerto 80 (HTTP) → Nginx → Redirect a 443
- |— Puerto 443 (HTTPS) → Nginx → Proxy reverso
- |— Puerto 22 (SSH) → Acceso administrativo

Interno:

- |— Puerto 8000 → Django Ecodisseny

- └─ Puerto 8001 → Django Oscar Shop
- └─ Puerto 8002 → Future App
- └─ Puerto 5432 → PostgreSQL

III FASE 8: MONITOREO Y MANTENIMIENTO

8.1 Scripts de Monitoreo

```
# Script de monitoreo general
cat > /home/deploy/scripts/monitor.sh << 'EOF'
#!/bin/bash
echo "=== 🖨️ ESTADO DEL SERVIDOR $(date) ==="
echo "📊 CPU y Memoria:"
echo "CPU: $(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)%
usado"
echo "RAM: $(free -h | awk 'NR==2{printf "%.1f/%.1fGB (%.1f%%)\n",
$3/1024/1024,$2/1024/1024,$3*100/$2}')"
echo "💾 Espacio: $(df -h / | tail -1 | awk '{print $3 "/" $2 " (" $5
")"}')'"

echo "🔧 Servicios:"
for service in nginx ecodisseny postgresql; do
    if systemctl is-active $service >/dev/null; then
        echo "✅ $service: ACTIVO"
    else
        echo "❌ $service: INACTIVO"
    fi
done

echo "🌐 Web: $(curl -s -o /dev/null -w "%{http_code}"
https://app.arasmu.net)"
EOF

chmod +x /home/deploy/scripts/monitor.sh
```

8.2 Backups Automatizados

```
# Script de backup completo
cat > /home/deploy/scripts/backup_completo.sh << 'EOF'
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/home/deploy/backups"

# Backup BD
pg_dump ecodisseny_prod > $BACKUP_DIR/ecodisseny_db_$DATE.sql
gzip $BACKUP_DIR/ecodisseny_db_$DATE.sql

# Backup media
```

```
tar -czf $BACKUP_DIR/ecodisseny_media_$DATE.tar.gz -C
/home/deploy/ecodisseny media/

# Limpiar antiguos (7 días)
find $BACKUP_DIR -name "ecodisseny_*" -mtime +7 -delete

echo "✓ Backup completado: $DATE"
EOF

chmod +x /home/deploy/scripts/backup_completo.sh

# Cron para backup diario
(crontab -l 2>/dev/null; echo "0 2 * * *
/home/deploy/scripts/backup_completo.sh") | crontab -
```

8.3 Scripts de Actualización

```
# Script de actualización
cat > /home/deploy/scripts/actualizar_app.sh << 'EOF'
#!/bin/bash
cd /home/deploy/ecodisseny

# Backup de seguridad
/home/deploy/scripts/backup_completo.sh

# Actualizar código
git pull origin docker
source venv/bin/activate
pip install -r requirements.txt

# Aplicar cambios
python manage.py migrate --settings=ecodisseny.settings_production
python manage.py cargar_documentacion --update --
settings=ecodisseny.settings_production
python manage.py collectstatic --noinput --
settings=ecodisseny.settings_production

# Reiniciar servicios
sudo systemctl restart ecodisseny
sudo systemctl reload nginx

echo "✓ Actualización completada"
EOF

chmod +x /home/deploy/scripts/actualizar_app.sh
```

🔍 FASE 8: TROUBLESHOOTING

8.1 Problemas Comunes con Docker

Error 502 Bad Gateway

```
# 1. Verificar contenedores
docker-compose ps

# 2. Ver logs de Django
docker-compose logs -f web

# 3. Verificar que Django responde internamente
curl -I http://localhost:8000

# 4. Reiniciar servicios
docker-compose restart web
sudo systemctl reload nginx
```

Contenedores no inician

```
# Ver logs detallados
docker-compose logs web
docker-compose logs db

# Reconstruir desde cero
docker-compose down -v # ⚠ ELIMINA VOLÚMENES
docker-compose build --no-cache
docker-compose up -d

# O usar el script de despliegue
./deploy-vps.sh
```

Error de Base de Datos

```
# Verificar container PostgreSQL
docker-compose exec db pg_isready -U ecodisseny

# Ver logs de PostgreSQL
docker-compose logs db

# Test de conexión desde Django
docker-compose exec web python manage.py dbshell
```

Problemas con archivos estáticos

```
# Recolectar archivos estáticos manualmente
docker-compose exec web python manage.py collectstatic --noinput
```

```
# Verificar volúmenes
docker volume ls
docker volume inspect ecodisseny_dj_pg_static_files

# Verificar permisos en Nginx
sudo ls -la /var/lib/docker/volumes/ecodisseny_dj_pg_static_files/_data/
```

8.2 Comandos de Debugging

```
# Monitoreo en tiempo real
docker-compose logs -f                # Todos los servicios
docker-compose logs -f web            # Solo Django
docker-compose logs -f db            # Solo PostgreSQL

# Acceso a containers
docker-compose exec web bash          # Shell en Django container
docker-compose exec db psql -U ecodisseny ecodisseny_db # PostgreSQL

# Información del sistema
docker system df                      # Uso de espacio Docker
docker-compose top                    # Procesos en containers
docker stats                          # Uso de recursos en tiempo real

# Verificar red
docker network ls
docker-compose port web 8000          # Puerto mapeado
```

8.3 Script de Diagnóstico Completo

```
# Script de diagnóstico automático
cat > /home/deploy/ecodisseny/scripts/diagnostico.sh << 'EOF'
#!/bin/bash
echo "🔍 DIAGNÓSTICO COMPLETO DEL SISTEMA"
echo "===== "

echo "📊 SISTEMA:"
echo "CPU: $(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)%"
echo "RAM: $(free -h | awk 'NR==2{printf "%.1f/%.1fGB (%.1f%%)\n",
$3/1024/1024,$2/1024/1024,$3*100/$2}')"
echo "Disco: $(df -h / | tail -1 | awk '{print $5}')"

echo ""
echo "🐳 DOCKER:"
docker --version
docker-compose --version
echo "Containers:"
docker-compose ps
```

```

echo ""
echo "🌐 CONECTIVIDAD:"
echo "Puerto 8000: $(curl -s -o /dev/null -w "%{http_code}"
http://localhost:8000 2>/dev/null || echo 'Error')"
echo "HTTPS: $(curl -s -o /dev/null -w "%{http_code}"
https://app.arasmu.net 2>/dev/null || echo 'Error/Sin SSL')"

echo ""
echo "🔧 SERVICIOS SISTEMA:"
for service in nginx docker; do
    if systemctl is-active $service >/dev/null; then
        echo "✅ $service: ACTIVO"
    else
        echo "❌ $service: INACTIVO"
    fi
done

echo ""
echo "📁 VOLÚMENES DOCKER:"
docker volume ls | grep ecodisseny

echo ""
echo "📖 LOGS RECIENTES (últimas 10 líneas):"
echo "--- Django ---"
docker-compose logs --tail=10 web
echo "--- PostgreSQL ---"
docker-compose logs --tail=10 db
EOF

chmod +x /home/deploy/ecodisseny/scripts/diagnostico.sh

```

8.4 Checklist de Verificación

- ✓ VPS Contabo contratado y funcionando
- ✓ Docker y docker-compose instalados
- ✓ Repositorio clonado
- ✓ Script deploy-vps.sh ejecutado exitosamente
- ✓ Contenedores Docker corriendo (docker-compose ps)
- ✓ Nginx configurado y activo
- ✓ DNS Cloudflare apuntando a VPS
- ✓ SSL Let's Encrypt funcionando
- ✓ Aplicación accesible vía HTTPS
- ✓ Backups automatizados configurados
- ✓ Scripts de monitoreo funcionando

🎉 ¡DESPLIEGUE COMPLETADO!

✓ Resultado Final:

- 🚀 **Aplicación Django** en <https://app.arasmu.net>
- 🐳 **Docker** con PostgreSQL y Django optimizados
- 🛡️ **HTTPS seguro** con certificados automáticos
- 💾 **Backups diarios** automatizados con volúmenes Docker
- 📊 **Monitoreo** del sistema y containers
- 🛠️ **Scripts** de despliegue y mantenimiento automatizados

🎯 URLs Finales:

🌐 Producción: <https://app.arasmu.net>
🌐 IP directa: <http://161.97.147.142> (redirige a HTTPS)
🔑 Admin: <https://app.arasmu.net/admin/>
📖 Docs: <https://app.arasmu.net/documentacion/>

👥 Usuarios Listos:

👑 ADMIN:

- mulastone / ecodisseny2024
- gonzalo / ecodisseny2024

👤 USUARIOS:

- sarah / ecodisseny2024
- pilar / ecodisseny2024
- santiago / ecodisseny2024
- roger / ecodisseny2024

🚀 Comandos Clave:

```
# Despliegue/Actualización
./deploy-vps.sh

# Monitoreo
./scripts/diagnostico.sh
docker-compose logs -f web

# Backup
./scripts/backup_docker.sh

# Reinicio rápido
docker-compose restart web
```

🎯 Próximos Pasos:

1. 🛒 Instalar Django Oscar Shop en puerto 8001
2. 📊 Configurar monitoreo avanzado (Grafana/Prometheus)

3.  CI/CD con GitHub Actions
4.  Notificaciones por email
5.  Personalización y branding

¡Tu aplicación está lista para producción con Docker! 🚀🔒🐳 `sudo systemctl status ecodisseny sudo journalctl -u ecodisseny -f`

Verificar Nginx

`sudo nginx -t sudo systemctl restart ecodisseny nginx`

```
#### Error de Base de Datos

```bash
Verificar PostgreSQL
sudo systemctl status postgresql

Test de conexión
python -c "
import psycopg2
conn = psycopg2.connect(
 host='localhost',
 database='ecodisseny_prod',
 user='deploy',
 password='tu_password'
)
print('✓ Conexión exitosa')
"
```

### Usar Scripts de Configuración

```
Si hay problemas, usar script de reinicio completo
./setup_complete.sh

O recargar fixtures y usuarios
./load_fixtures.sh
```

## 9.2 Debugging Avanzado

```
Ver logs en tiempo real
sudo tail -f /var/log/nginx/app.arasmu.net.error.log \
 /var/log/django/ecodisseny.log

Verificar procesos
ps aux | grep gunicorn
```

```
sudo netstat -tlnp | grep :8000

Verificar espacio
df -h
free -h
```

### 9.3 Checklist de Verificación

- ✓ VPS Contabo contratado y configurado
- ✓ Usuario deploy con permisos sudo
- ✓ Firewall configurado (22, 80, 443)
- ✓ PostgreSQL funcionando
- ✓ Repositorio clonado
- ✓ Scripts setup\_complete.sh ejecutado
- ✓ Usuarios y perfiles creados
- ✓ Nginx configurado
- ✓ DNS Cloudflare configurado
- ✓ SSL Let's Encrypt funcionando
- ✓ Aplicación accesible vía HTTPS
- ✓ VSCode SSH configurado
- ✓ Backups automatizados
- ✓ Scripts de monitoreo funcionando

---

## 🎉 ¡DESPLIEGUE COMPLETADO!

### ✓ Resultado Final:

- 🚀 **Aplicación Django** en <https://app.arasmu.net>
- 🛡️ **HTTPS seguro** con certificados automáticos
- 💾 **Backups diarios** automatizados
- 📊 **Monitoreo** del sistema
- 💻 **VSCode remoto** para desarrollo
- 🛒 **Base preparada** para Oscar Shop

### 👥 Usuarios Listos:

- 👑 **ADMIN:**
- mulastone / ecodisseny2024
  - gonzalo / ecodisseny2024
- 👤 **USUARIOS:**
- sarah / ecodisseny2024
  - pilar / ecodisseny2024
  - santiago / ecodisseny2024
  - roger / ecodisseny2024

## Próximos Pasos:

1.  Instalar Django Oscar Shop (puerto 8001)
2.  Configurar monitoreo avanzado
3.  CI/CD para despliegues automáticos
4.  Notificaciones por email
5.  Personalización y branding

**¡Tu aplicación está lista para producción! 🚀**